

Recap

- PCA used for?
- PCA supervised or unsupervised?

Supervised dimensionality reduction - Forward feature Selection

- Start with **no features**.
- Add features **one at a time** — at each step, choose the feature that gives the biggest improvement to model performance (e.g., accuracy, R^2 , AIC).
- Stop when adding new features no longer improves performance beyond threshold.

Backward Feature Selection

- Start with **all features**.
- Remove features **one at a time** — at each step, drop the feature whose removal least reduces model performance.
- Stop when removing any more features harms performance too much.

Both forward and backward feature selection are quite computationally expensive

Linear Regression

Nimisha Roy
Georgia Tech

Outline

- Supervised Learning 
- Linear Regression
- Extension

Supervised Learning: Overview

Functions $\mathcal{F}$

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Training data

$$\{(x^{i}, y^{i}) \in \mathcal{X} \times \mathcal{Y}\}$$

LEARNING

$$\begin{array}{l} \text{find } \hat{f} \in \mathcal{F} \\ \text{s.t. } y_a \approx \hat{f}(x^{i}) \end{array}$$



Learning machine

PREDICTION

$$\hat{y}_p = \hat{f}(x^{i})$$

New data

$$x$$

Supervised Learning: Split of data

Training Set

- **Purpose:** Used to **fit** the model — the algorithm learns patterns, relationships, and weights from this data.
- **What happens here:** The model adjusts its internal parameters to minimize error.
- **Analogy:** Like studying the material before an exam.

Validation Set

- **Purpose:** Used to **tune hyperparameters** and make design decisions (e.g., how many layers, what learning rate, etc.).
- **What happens here:** The model is not trained on this data; instead, it helps to evaluate how changes affect performance, preventing overfitting.
- **Analogy:** Like taking practice tests to decide which study strategy works best.

Test Set

- **Purpose:** Used to **assess the final performance** of the model after all training and tuning.
- **What happens here:** No further learning or tweaking — it provides an **unbiased estimate** of the model's generalization ability.
- **Analogy:** Like the final exam — measures how well you really learned the material.

Supervised Learning: Two Types of Tasks

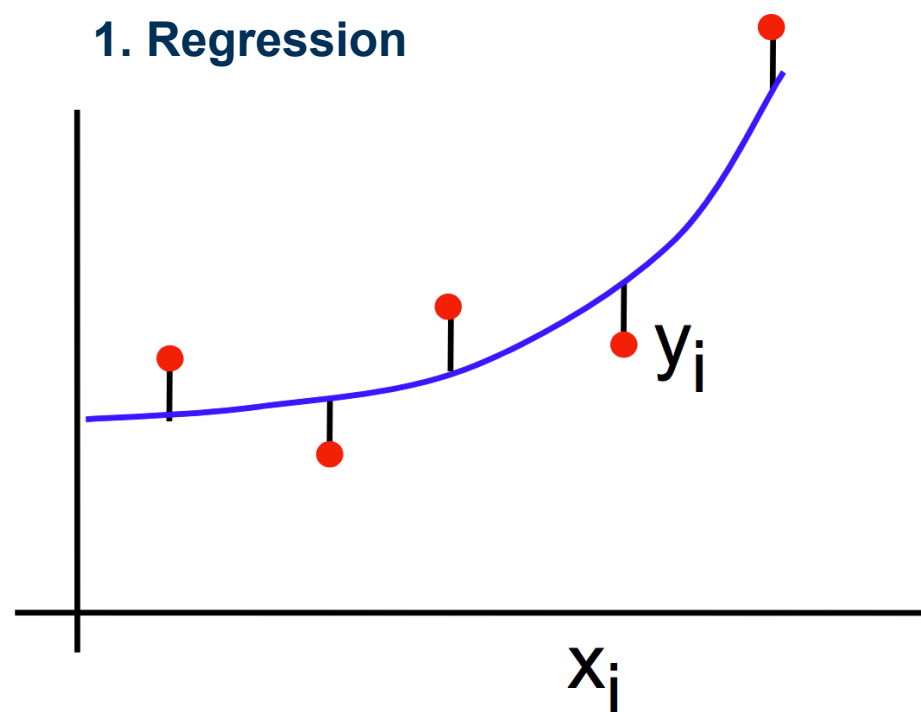
Given: training data

$$\{(x^{\{1\}}, y^{\{1\}}), (x^{\{2\}}, y^{\{2\}}), \dots, (x^{\{n\}}, y^{\{n\}})\}$$

Learn: a function

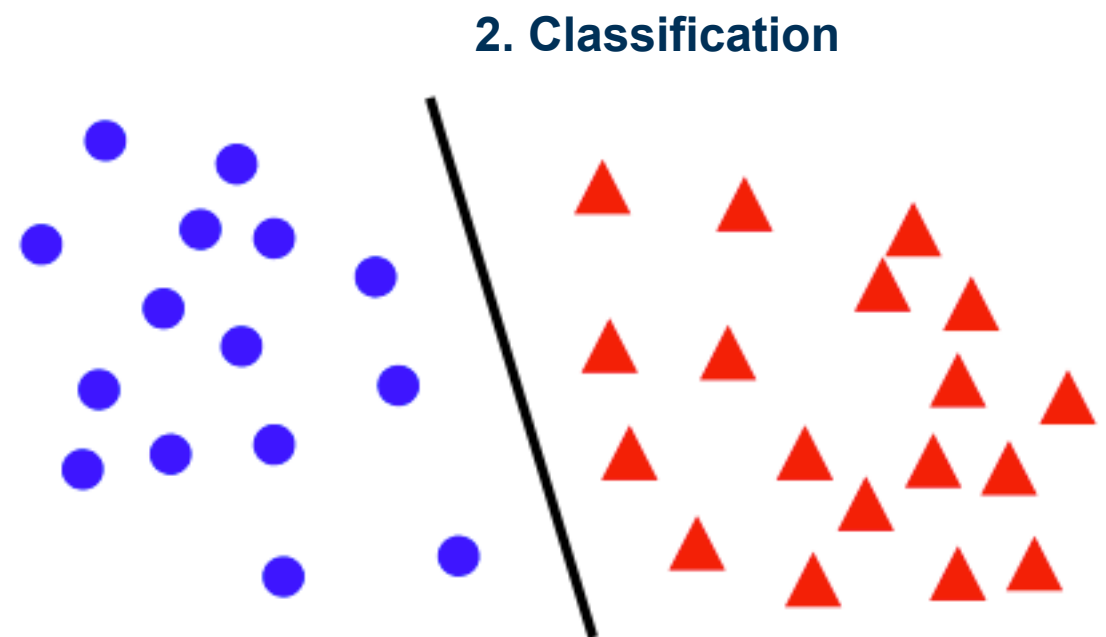
$$f(\mathbf{x}) : y = f(\mathbf{x})$$

When y is continuous:



Curve fitting

When y is discrete:

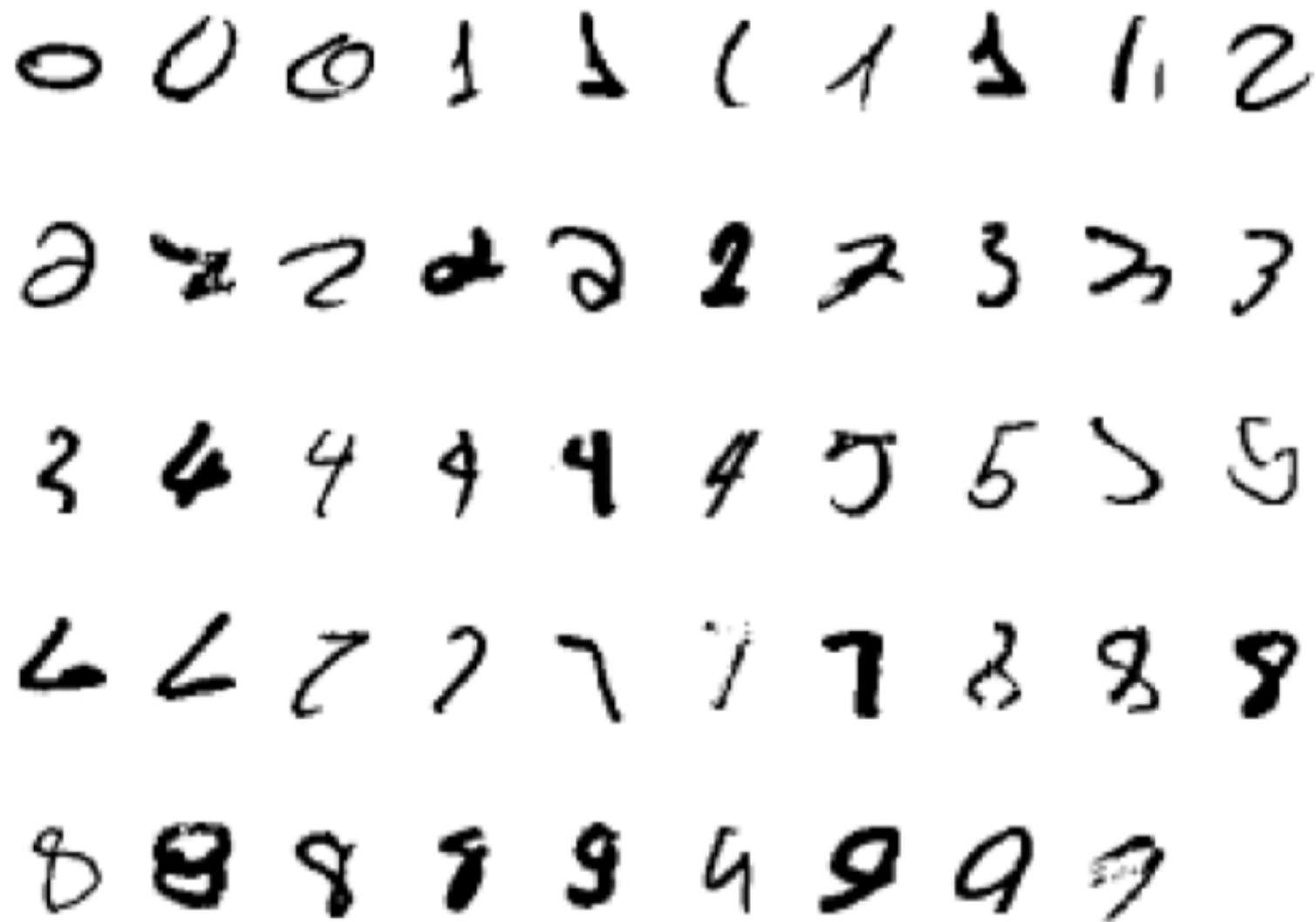


Class estimation

Classification Example 1: Handwritten digit recognition

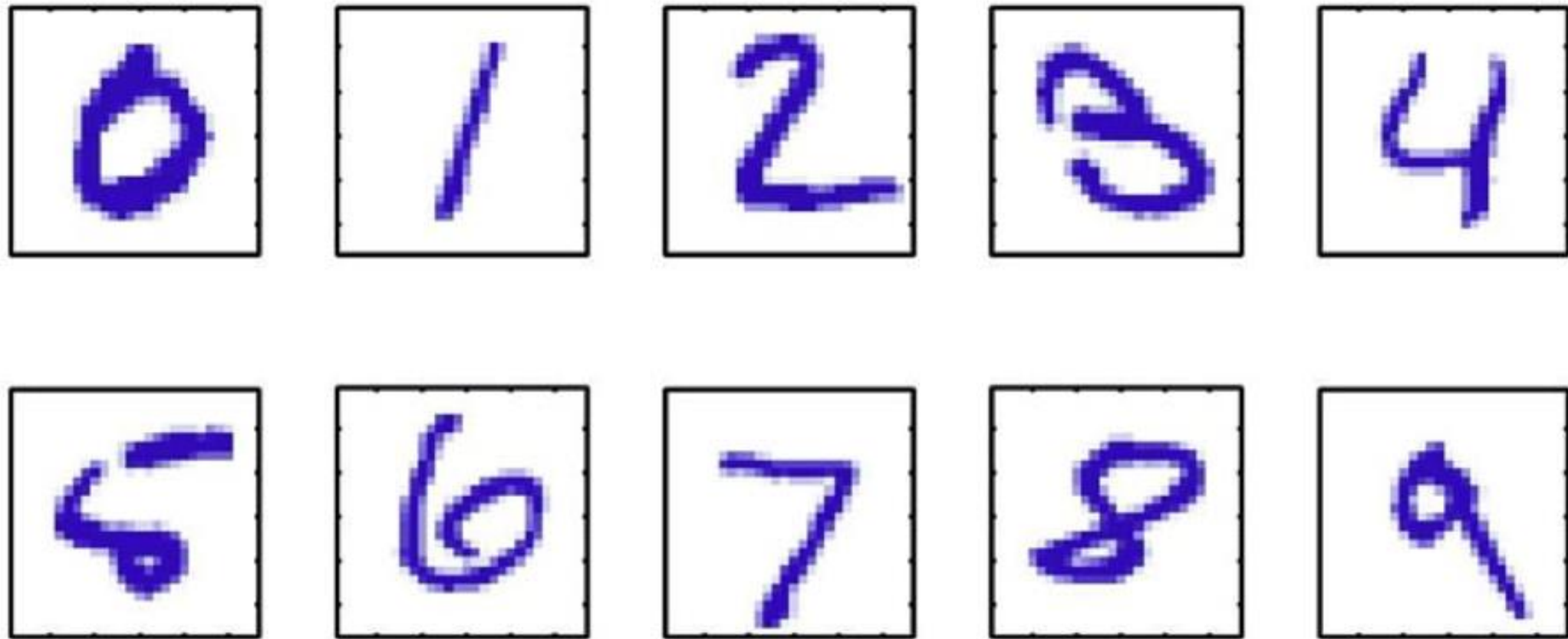
As a supervised classification problem

Start with training data, e.g. 6000 examples of each digit



- Can achieve testing error of 0.4%
- One of first commercial and widely used ML systems (for zip codes & checks)

Classification Example 1: Hand-Written Digit Recognition



Images are 28 x 28 pixels

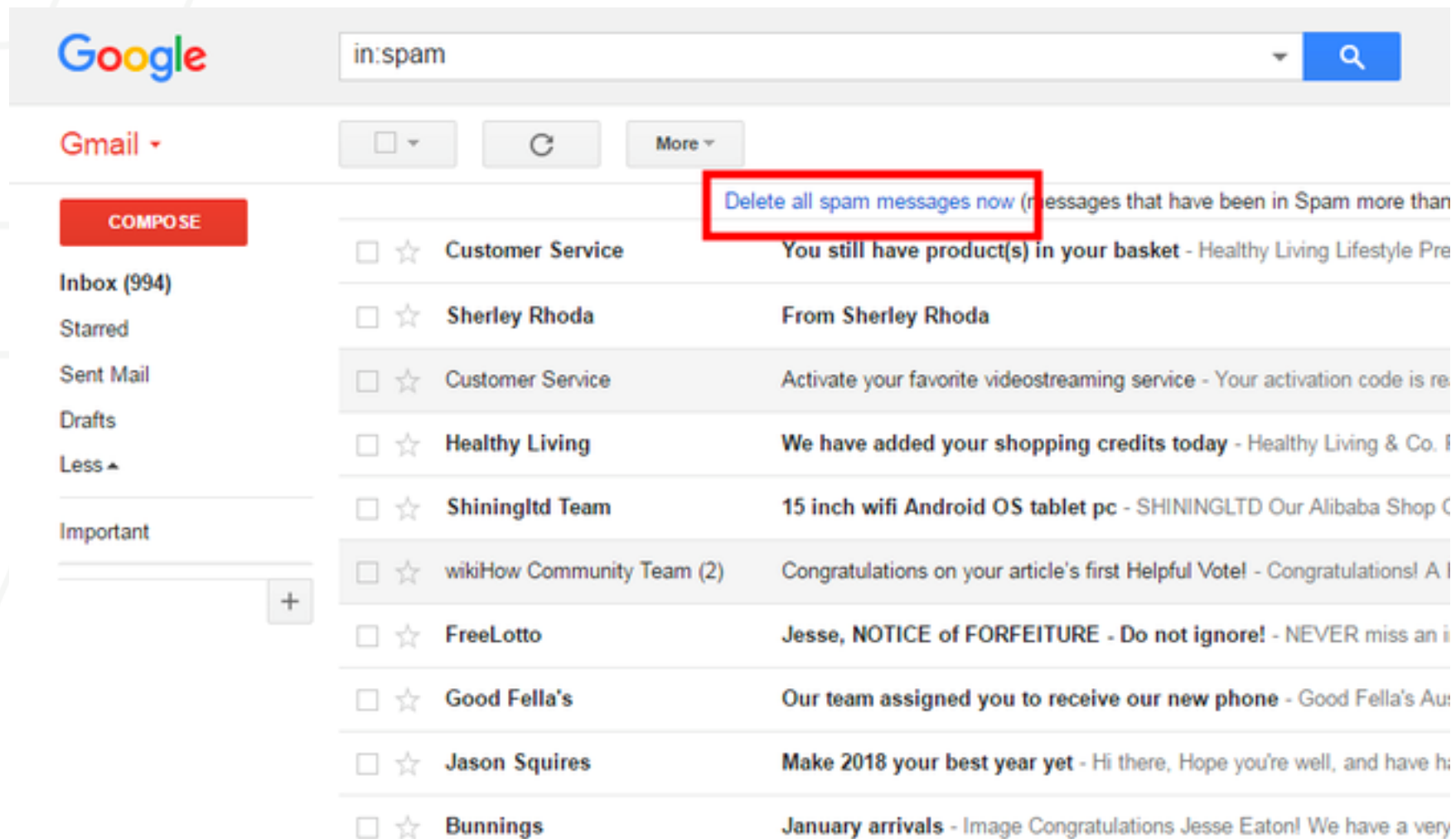
A classification problem

Represent input image as a vector $\mathbf{x} \in \mathbb{R}^{784}$

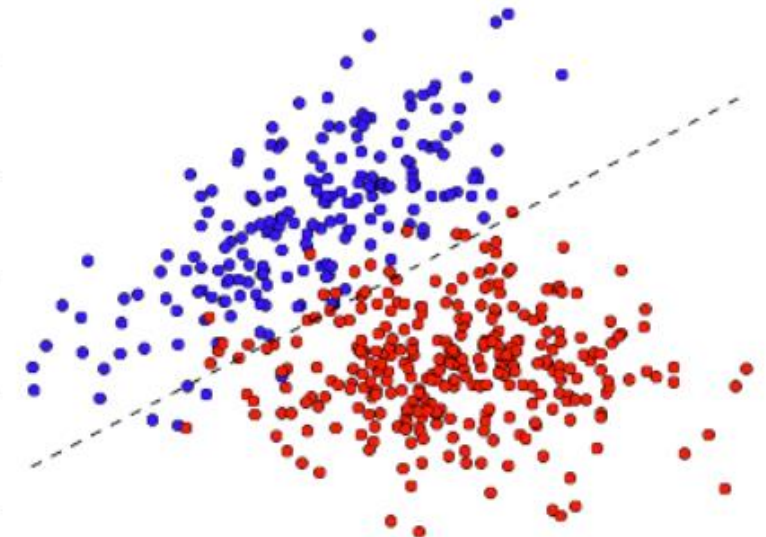
Learn a classifier $f(\mathbf{x})$ such that,

$$f : \mathbf{x} \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Classification Example 2: Spam Detection



NOT SPAM



SPAM

A classification problem

- This is a classification problem
- Task is to classify email into spam/non-spam
- Data x_i is word count
- Requires a learning system as “enemy” keeps innovating

Regression Example 1: Apartment Rent Prediction

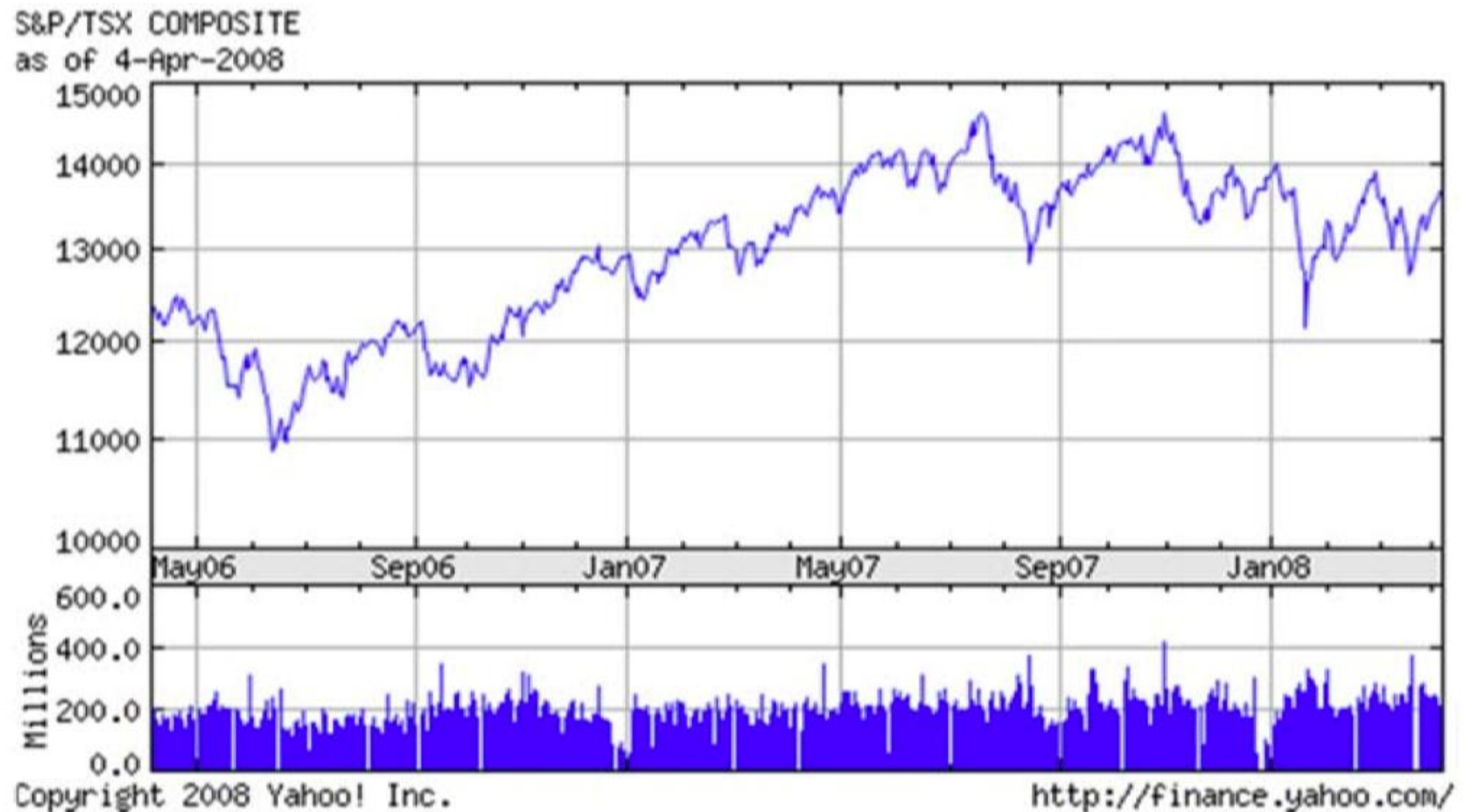
- Suppose you are to move to Atlanta
- And you want to find the **most reasonably priced** apartment satisfying your **needs**:

square-ft., # of bedroom, distance to campus ...

A regression problem

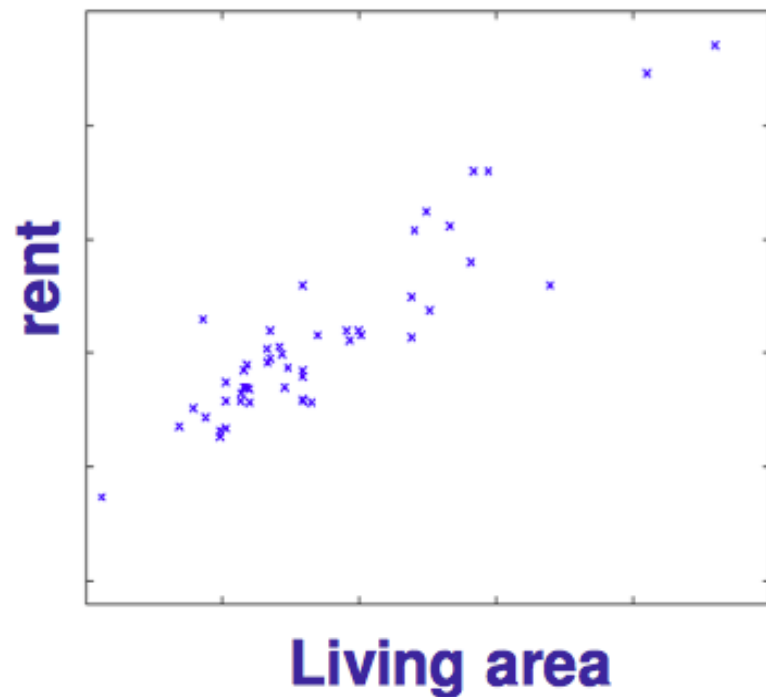
Living area (ft ²)	# bedroom	Rent (\$)
230	1	600
506	2	1000
433	2	1100
109	1	500
...		
150	1	?
270	1.5	?

Regression Example 2: Stock Price Prediction

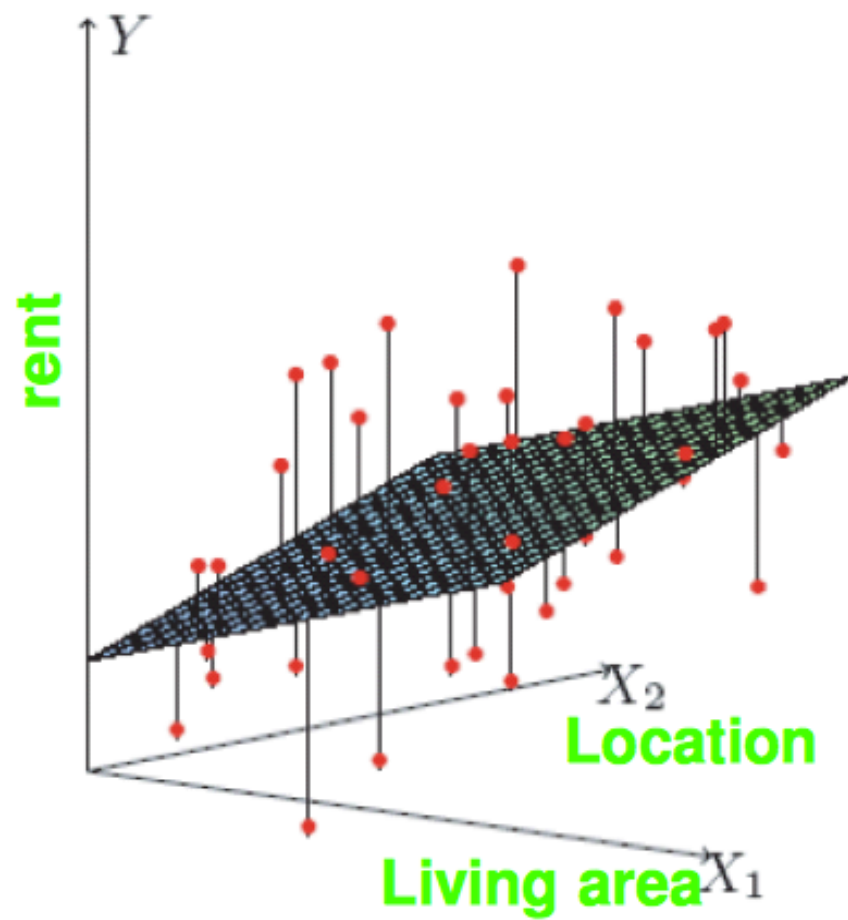


- Task is to predict stock price at future date

A regression problem



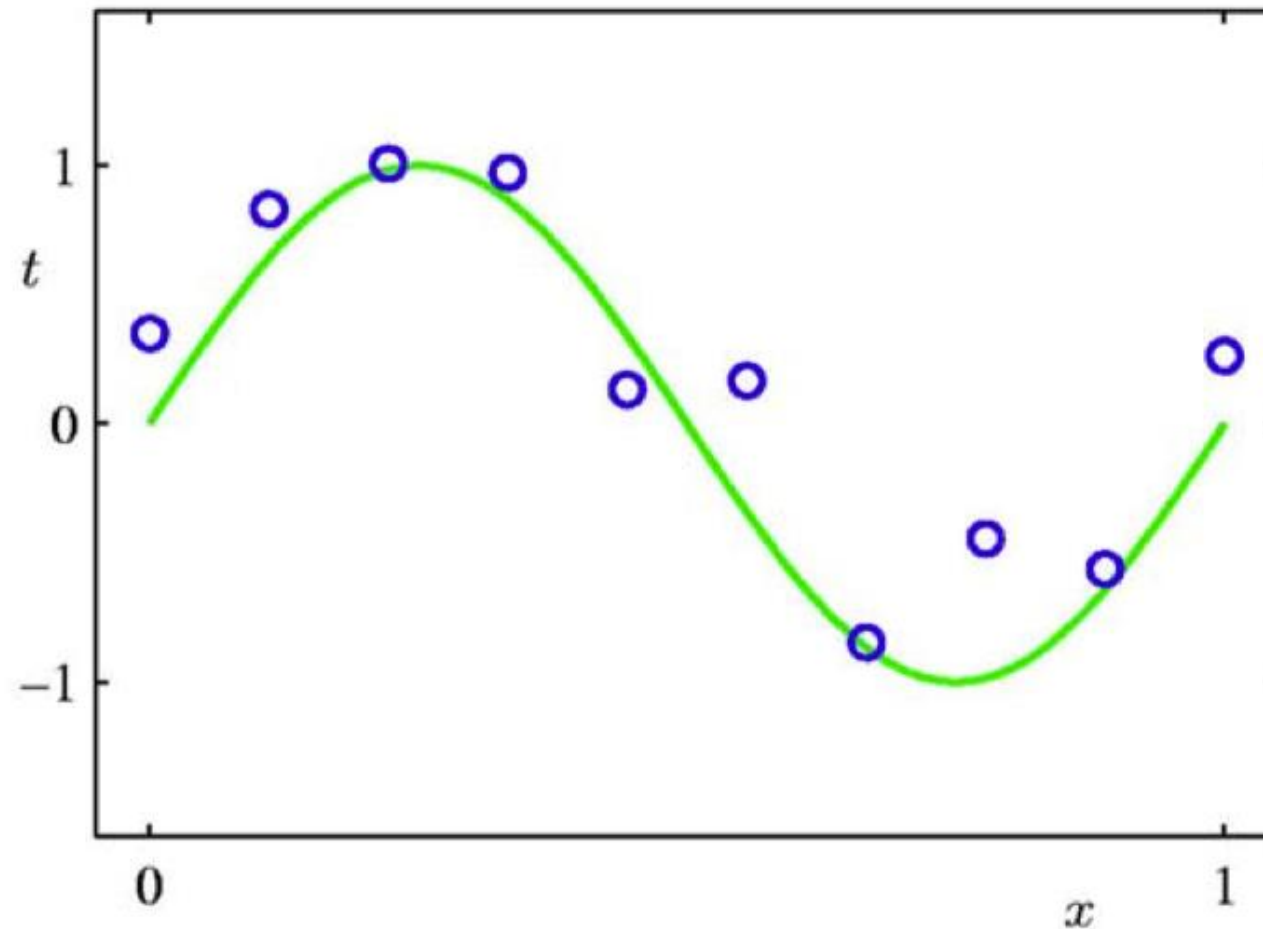
- Features:
 - Living area, distance to campus, # bedroom ...
 - Denote as $x = (x_1, x_2, \dots, x_d)$
- Target:
 - Rent
 - Denoted as y



- Training set:

- $x = \{x^{\{1\}}, x^{\{2\}}, \dots, x^{\{n\}}\} \in R^d$
- $y = \{y^{\{1\}}, y^{\{2\}}, \dots, y^{\{n\}}\}$

Regression: Problem Setup



- Suppose we are given a training set of N observations
- Regression problem is to estimate $y(x)$ from this data

Outline

- Supervised Learning
- Linear Regression 
- Extension

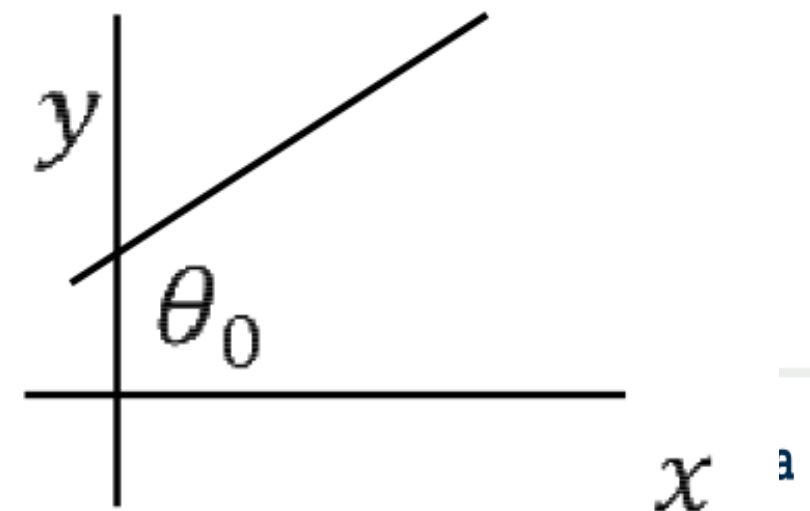
Linear Regression

- Assume y is a linear function of x (features) plus noise ϵ

$$y = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d + \epsilon$$

- where ϵ is an error term of unmodeled effects or [random noise](#)
- Let $\theta = (\theta_0, \theta_1, \dots, \theta_d)^\top$, and augment data by one dimension

- Then $y = x\theta + \epsilon$



Least Mean Square Method

- Given n data points, find θ that minimizes the mean square error

Training $\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{n} \sum_{i=1}^N (y^{\{i\}} - x^{\{i\}} \theta)^2$

- Our usual trick: set gradient to 0 and find parameter $\frac{\partial L(\theta)}{\partial \theta} = 0$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}} \theta) = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T y^{\{i\}} + \frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T x^{\{i\}} \theta = 0$$

Matrix form

$$x = \begin{bmatrix} 1 & x_1^{\{1\}} & \dots & x_d^{\{1\}} \\ 1 & x_1^{\{2\}} & \ddots & x_d^{\{2\}} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{\{n\}} & \dots & x_d^{\{n\}} \end{bmatrix}_{n \times (d+1)} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{(d+1) \times 1}$$

$$MSE(\theta) = \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{n} (y - x\theta)^T (y - x\theta)$$

$$x\theta = \begin{bmatrix} \theta_0 + \theta_1 x_1^{\{1\}} + \theta_2 x_2^{\{1\}} + \dots + \theta_d x_d^{\{1\}} \\ \theta_0 + \theta_1 x_1^{\{2\}} + \theta_2 x_2^{\{2\}} + \dots + \theta_d x_d^{\{2\}} \\ \vdots \\ \theta_0 + \theta_1 x_1^{\{n\}} + \theta_2 x_2^{\{n\}} + \dots + \theta_d x_d^{\{n\}} \end{bmatrix}_{n \times 1}$$

Matrix Version and Optimization

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T y^{\{i\}} + \frac{2}{n} \sum_{i=1}^N (x^{\{i\}})^T x^{\{i\}} \theta = 0$$

Let's rewrite it as:

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} (x^{\{1\}}, \dots, x^{\{n\}})^T (y^{\{1\}}, \dots, y^{\{n\}}) + \frac{2}{n} (x^{\{1\}}, \dots, x^{\{n\}})^T (x^{\{1\}}, \dots, x^{\{n\}}) \theta = 0$$

Define $X = (x^{\{1\}}, \dots, x^{\{n\}})$ *and* $y = (y^{\{1\}}, \dots, y^{\{n\}})$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} X^T y + \frac{2}{n} X^T X \theta = 0$$

X^+ is the pseudo-inverse of X

$$\Rightarrow \theta = (X^T X)^{-1} X^T y = X^+ y$$

$$X^T X X^+ = X^T$$

$$\theta = (X^T X)^{-1} X^T y = X^+ y$$

$X_{n \times d}$

$n = \text{instances}$

$d = \text{dimension}$

$$X^T X = \left[\begin{array}{c} d \times n \\ n \times d \end{array} \right] = \left[\begin{array}{c} d \times d \end{array} \right]$$

Not a big matrix because $n \gg d$ This matrix is invertible most of the times.
 If we are VERY unlucky and columns of $X^T X$ are not linearly independent (it's not a full rank matrix), then it is not invertible.

Quick Knowledge Check